INTERNSHIP REPORT

WEEK 02

NETWORKING FUNDAMENTALS

Submitted by

SAIF SAMI

Internship Domain : Cybersecurity

Internship Organization

NEXT GEN SOFTWARE HUB

Academic Institution

ORIENTAL COLLEGE OF TECHNOLOGY

Report Period

Week 02

Submission date

18 October, 2026

TABLE OF CONTENTS

1. **Introduction**
2. **Objectives**
3. **Networking Concepts**
   - **3.1 OSI Model**
   - **3.2 TCP/IP Model**
   - **3.3 IP Addressing**
   - **3.4 Domain Name System (DNS)**
   - **3.5 Dynamic Host Configuration Protocol (DHCP)**
4. **Tools Used**
   - **4.1 Kali Linux**
   - **4.2 Wireshark**
   - **4.3 Curl**
   - **4.4 Practical Commands**
5. **Practical Procedure**
   - **5.1 Network Interface Identification**
   - **5.2 Connectivity Verification**
   - **5.3 Starting Wireshark**
   - **5.4 HTTP Traffic Analysis**
   - **5.5 HTTPS/TLS Traffic Analysis**
   - **5.6 Saving the Capture**
6. **HTTP Traffic Analysis**
7. **HTTPS/TLS Traffic Analysis**
8. **Findings and Learning Outcomes**
9. **Conclusion**
10. **References**
11. **Appendix – Screenshots and Evidence**

1. Introduction

Networking is an essential part of cybersecurity because computers and other devices communicate through networks to exchange information and access services. Understanding network communication helps cybersecurity professionals identify suspicious traffic, troubleshoot connectivity issues, and understand how protocols operate.

This practical was performed as part of the Cybersecurity Internship at Next Gen Software Hub Pvt. Ltd. The main focus was to use Kali Linux and Wireshark to capture network packets and examine HTTP and HTTPS/TLS traffic.

Wireshark was used to observe packets exchanged over the network and inspect available protocol information. The practical helped develop an understanding of packet capture, protocol filtering, and the differences between unencrypted HTTP communication and encrypted HTTPS communication.

2. Objectives

The objectives of this practical were:

1. To understand the basic concepts of computer networking.
2. To identify the purpose of the OSI and TCP/IP models.
3. To understand IP addressing, DNS, and DHCP.
4. To capture network packets using Wireshark.
5. To identify and analyze HTTP traffic using display filters.
6. To capture and inspect HTTPS/TLS traffic.
7. To understand how encryption protects data transmitted over HTTPS.
8. To document the practical procedure and findings with screenshots.

3. Networking Concepts

3.1 OSI Model

The Open Systems Interconnection (OSI) model is a seven-layer reference model that explains how data travels between networked devices. Each layer performs a specific function in network communication.

Layer	Name	Main Function
7	Application	Provides network services to applications
6	Presentation	Handles data formatting, encryption, and compression
5	Session	Establishes and manages communication sessions
4	Transport	Provides end-to-end transport using TCP or UDP
3	Network	Handles logical addressing and routing
2	Data Link	Handles frames and MAC addressing
1	Physical	Transmits raw bits through physical or wireless media

The OSI model helps network administrators and cybersecurity professionals understand communication processes and troubleshoot network issues.



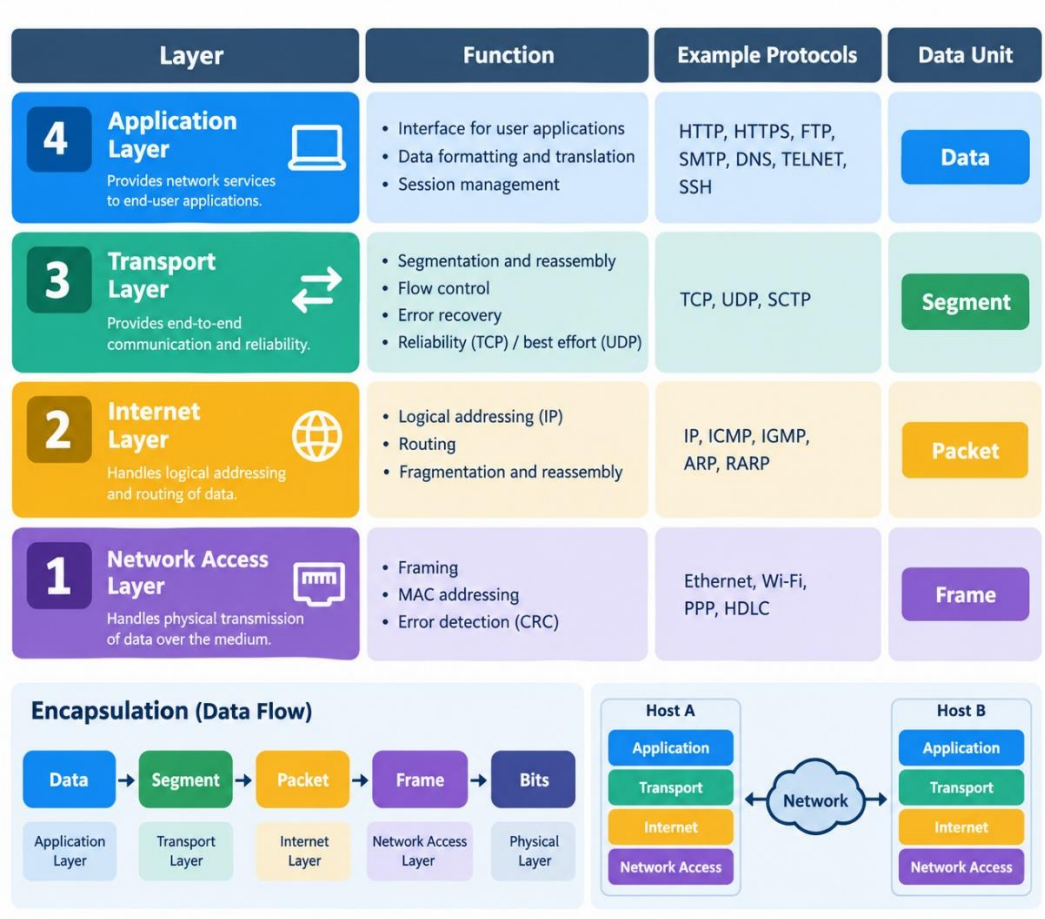
OSI Model Diagram

3.2 TCP/IP Model

The TCP/IP model describes the protocols and functions used for communication over modern networks and the internet. It is commonly represented using four layers.

Layer	Main Function	Examples
Application	Provides application-level network services	HTTP, HTTPS, DNS, DHCP
Transport	Supports communication between endpoints	TCP, UDP
Internet	Provides addressing and routing	IP, ICMP
Network Access	Handles local network delivery and physical transmission	Ethernet, Wi-Fi

The OSI model has seven layers, whereas the commonly used TCP/IP model has four. Both models help explain network communication, but they organize networking functions differently.



TCP/IP Diagram

3.3 IP Addressing

An Internet Protocol (IP) address is a logical address used to identify a device or network interface and support communication across IP networks.

IPv4 addresses are 32 bits long and are commonly written as four decimal numbers separated by dots, such as 192.168.1.10.

IP addresses can be public or private. Private IPv4 address ranges are used inside local networks, while public addresses are used for communication across the public internet.

3.4 DNS

The Domain Name System (DNS) translates domain names, such as example.com, into IP addresses used by networked devices. DNS makes it easier for users to access internet services without memorizing numerical IP addresses.

3.5 DHCP

Dynamic Host Configuration Protocol (DHCP) automatically provides network configuration information to devices. This information commonly includes an IP address, subnet mask, default gateway, and DNS server addresses.

DNS helps resolve domain names, while DHCP helps devices obtain network configuration automatically.

4. Tools Used

4.1 Kali Linux

Kali Linux was used as the operating system for conducting the networking practical. The terminal was used to check network interface configuration, verify connectivity, and generate test network traffic.

4.2 Wireshark

Wireshark is a network protocol analyzer that captures and displays packets transmitted through a selected network interface. It provides packet details that help users understand network communication and investigate network issues.

In this practical, Wireshark was used to capture network traffic and apply display filters to examine HTTP and HTTPS/TLS packets.

4.3 Curl

The curl command-line utility was used to generate an HTTPS request to https://example.com. This generated traffic that could be examined in Wireshark.

4.4 Practical Commands

The following commands were used during the practical:

Command	Purpose
ip addr	Displays network interfaces and IP addresses
ping -c 4 192.168.52.2	Tests connectivity to the specified address
curl -I https://example.com	Sends a request for HTTP response headers over HTTPS
tls	Wireshark display filter for recognized TLS traffic
tcp.port == 443	Displays TCP packets using port 443

The ping command assumes that 192.168.52.2 is the configured and reachable target in the lab network.

5. Practical Procedure

Step 1: Network Interface Identification

The ip addr command was executed in Kali Linux to identify the active network interface and its assigned IP address.

Evidence:

```
C:\home\kali> ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1
    link/ether 00:0c:29:50:f9:47 brd ff:ff:ff:ff:ff:ff
    inet 192.168.52.128/24 brd 192.168.52.255 scope global dynamic noprefixroute eth0
       valid_lft 1551sec preferred_lft 1551sec
    inet6 fe80::4d34:f7ce:62be:3e1f/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 7e:10:cd:e0:8e:5e brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
```

Step 2: Connectivity Verification

The command ping -c 4 192.168.52.2 was used to test connectivity to the configured lab network address. The result was checked in the terminal.

Evidence:

```
C:\home\kali> ping -c 4 192.168.52.128
PING 192.168.52.128 (192.168.52.128) 56(84) bytes of data.
64 bytes from 192.168.52.128: icmp_seq=1 ttl=64 time=0.283 ms
64 bytes from 192.168.52.128: icmp_seq=2 ttl=64 time=0.151 ms
64 bytes from 192.168.52.128: icmp_seq=3 ttl=64 time=0.135 ms
64 bytes from 192.168.52.128: icmp_seq=4 ttl=64 time=0.146 ms

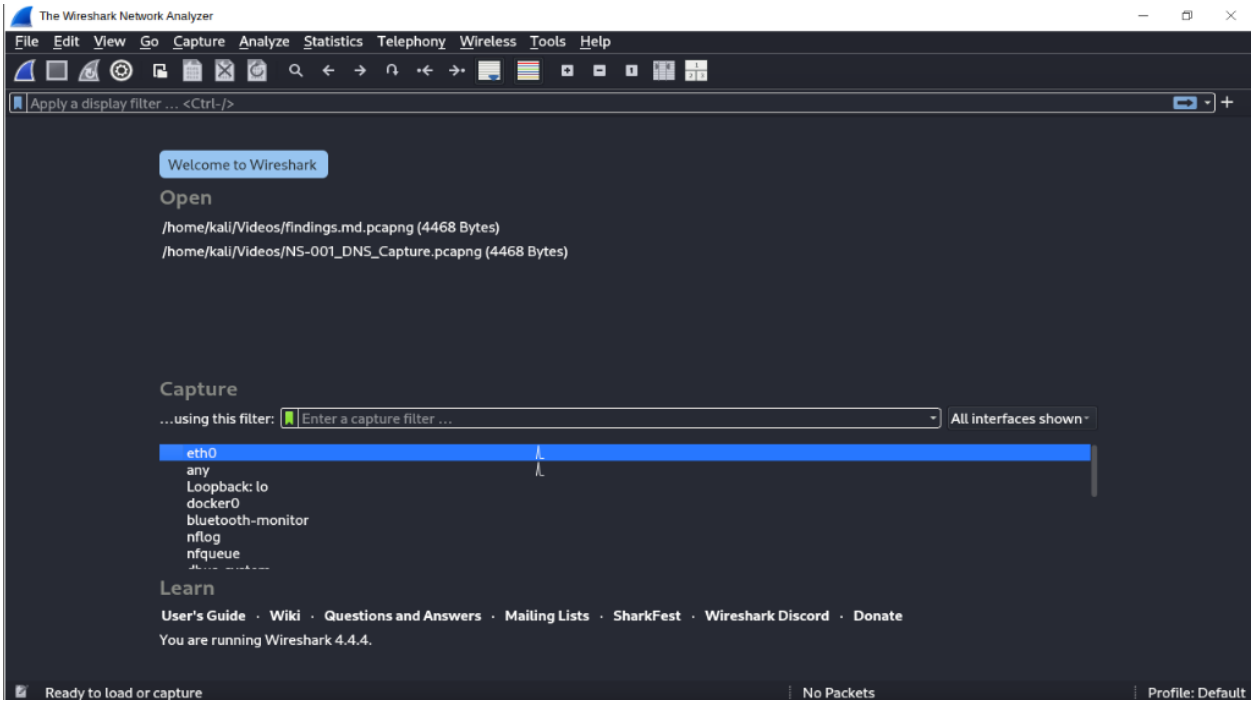
── 192.168.52.128 ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3073ms
rtt min/avg/max/mdev = 0.135/0.178/0.283/0.060 ms
```

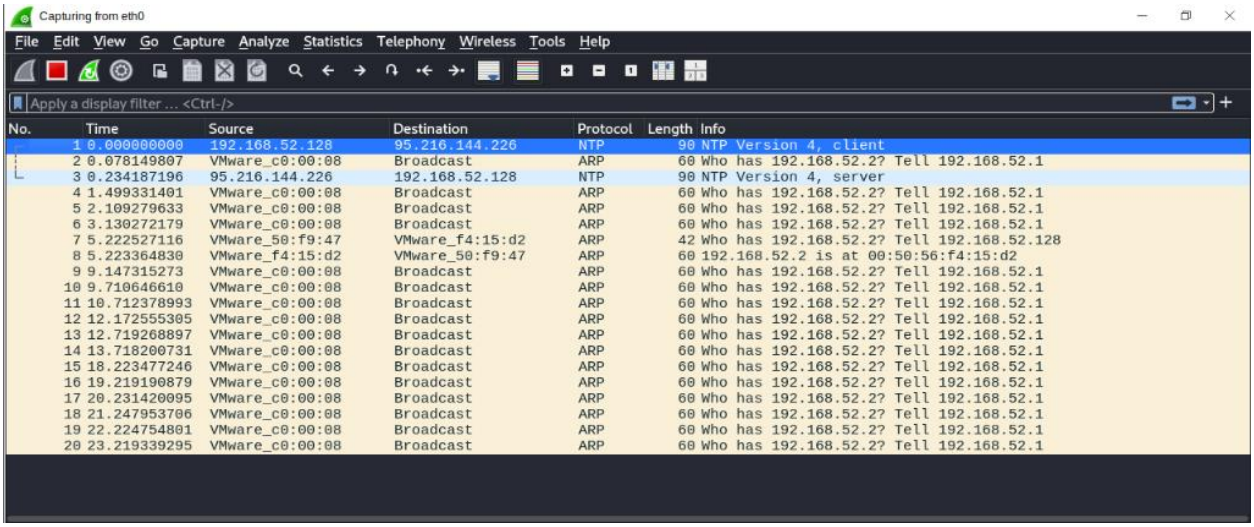
Step 3: Starting Wireshark

Wireshark was opened, and the active network interface was selected. A packet capture was started to observe live network traffic.

Evidence:

```
C:\home\kali> wireshark
 ** (wireshark:4771) 00:43:56.138672 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::p
alette
 ** (wireshark:4771) 00:43:56.147343 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::p
tonPalette
 ** (wireshark:4771) 00:43:56.147560 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::p
alette
 ** (wireshark:4771) 00:43:56.148074 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::p
xPalette
 ** (wireshark:4771) 00:43:56.148308 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::p
ttonPalette
```

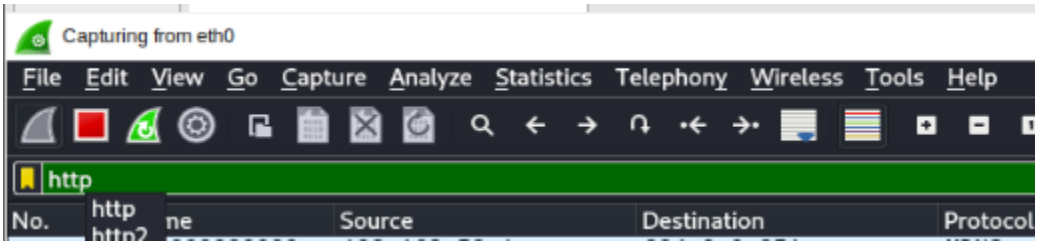


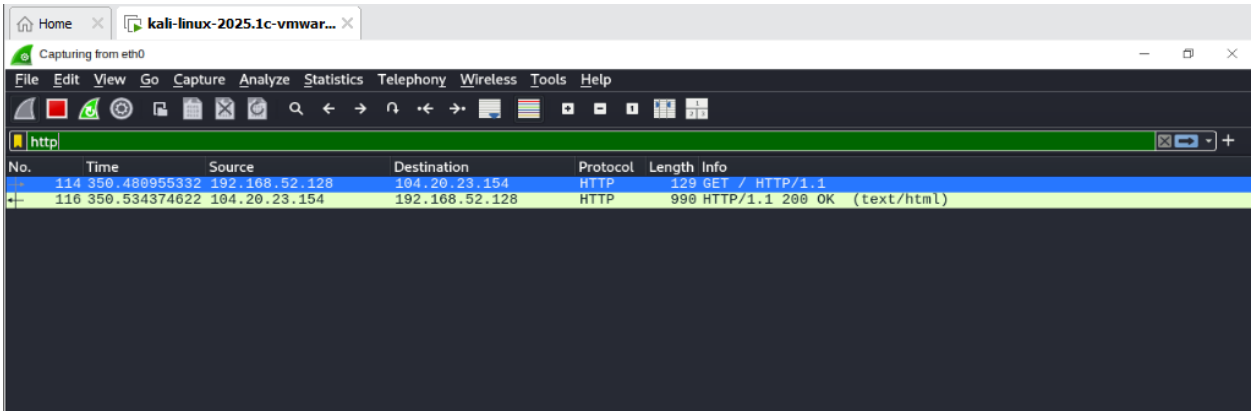


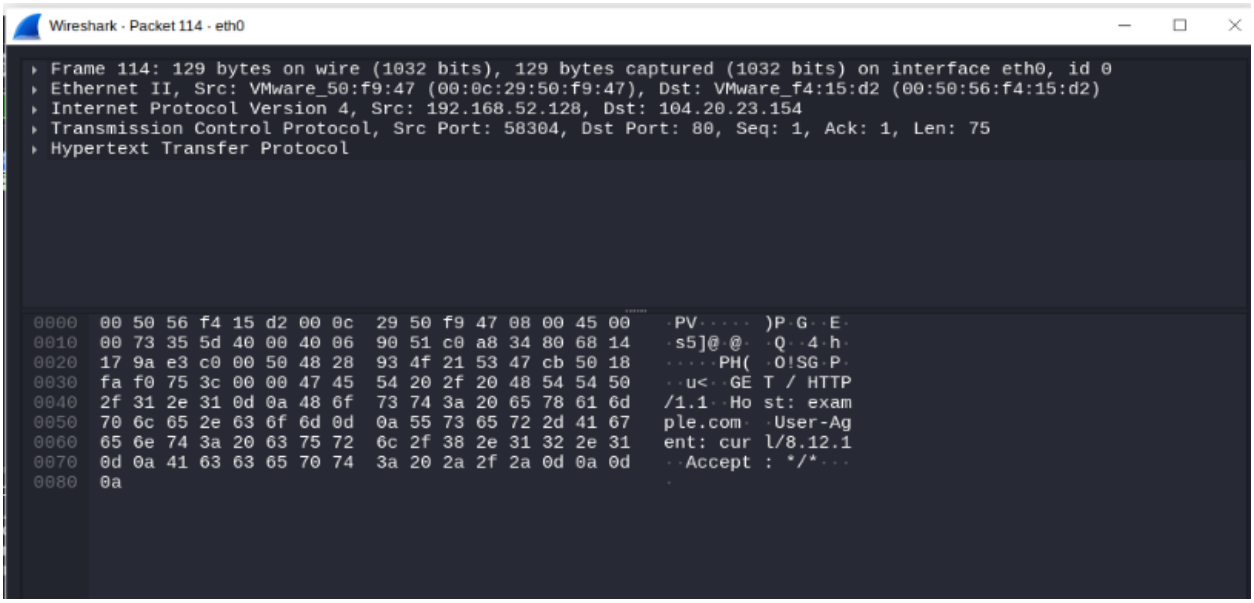
Step 4: HTTP Traffic Analysis

The HTTP display filter was applied to inspect captured HTTP packets. Relevant packet details were examined where available.

Evidence:



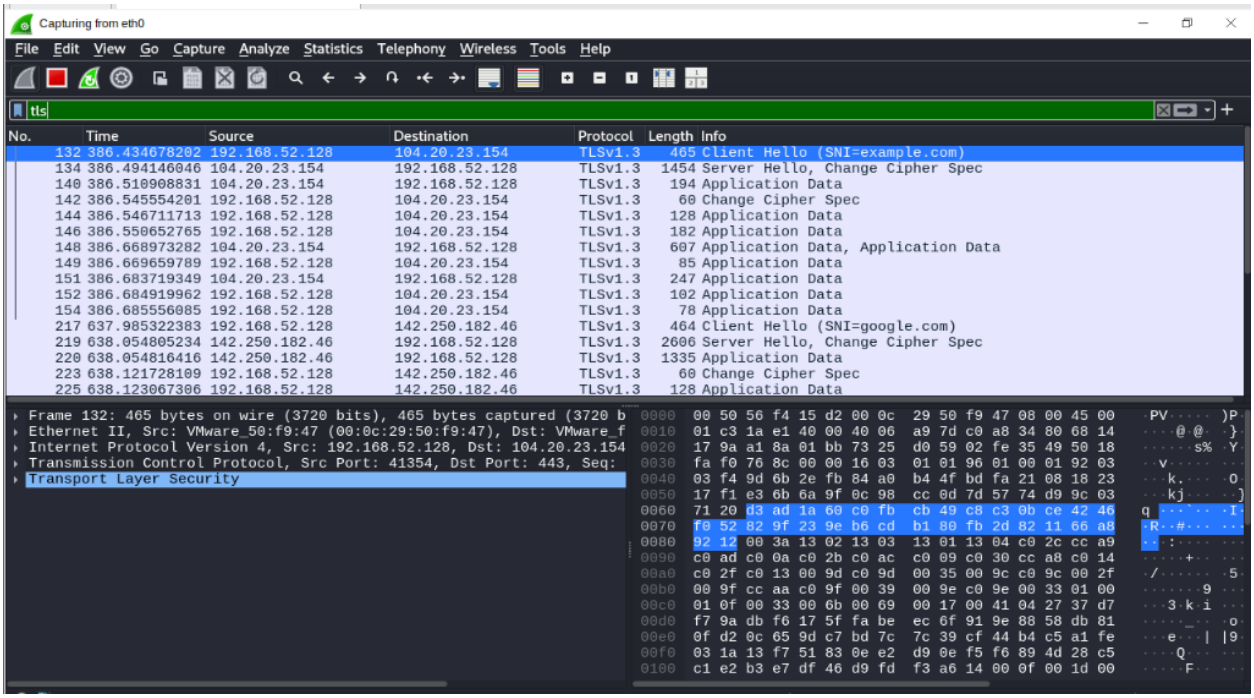




Step 5: HTTPS/TLS Traffic Analysis

An HTTPS request was generated using curl -I https://example.com. The tls display filter was then applied to identify TLS packets. The filter tcp.port == 443 was also available to inspect TCP traffic on the standard HTTPS port.

Evidence:



Step 6: Saving the Capture

The packet capture was saved in PCAPNG format for future inspection and documentation.

Evidence: Include the saved capture file in the Week 2 Evidence folder.

6. HTTP Traffic Analysis

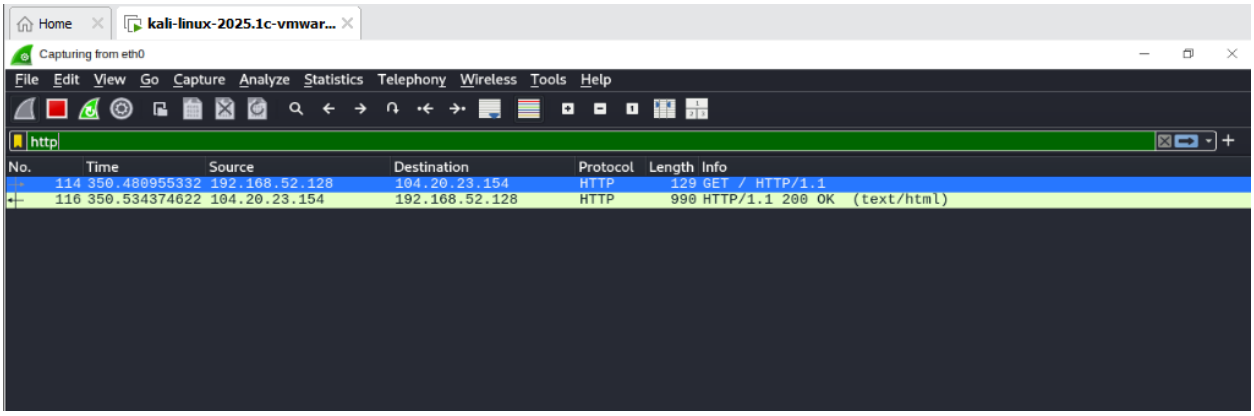
HTTP (Hypertext Transfer Protocol) is used for communication between web clients and servers. Standard HTTP traffic commonly uses TCP port 80.

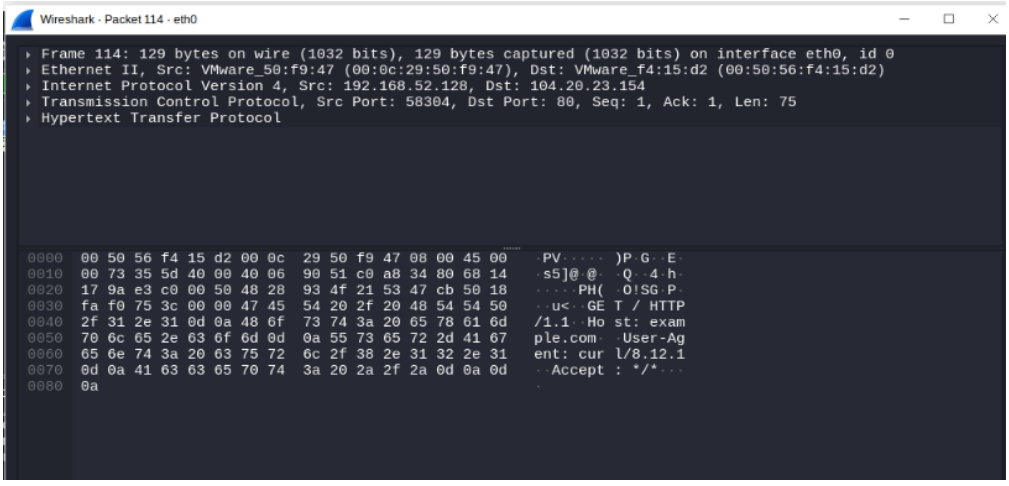
During the practical, Wireshark was used to capture and inspect HTTP packets. The http display filter was applied to identify packets recognized as HTTP traffic. Available packet details were examined to understand the communication between the client and server.

Observations

- Wireshark provided a packet-level view of captured HTTP traffic.
- The HTTP display filter helped isolate HTTP packets from other captured protocols.
- Depending on the captured request, packet details could include the request method, host, request URI, and response status.
- Ordinary HTTP does not encrypt application data, so sensitive information transmitted through HTTP may be exposed to someone able to observe the traffic.

Evidence: Figure 4 shows the HTTP traffic captured during the practical.





7. HTTPS/TLS Traffic Analysis

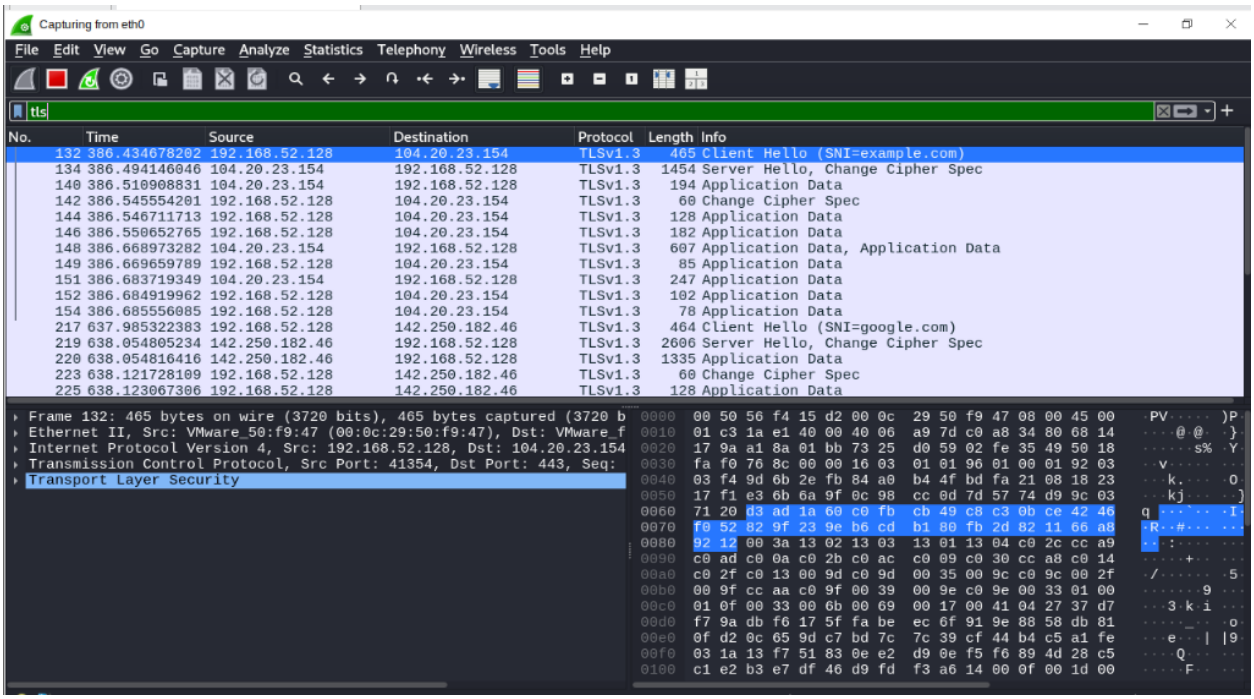
HTTPS (Hypertext Transfer Protocol Secure) protects HTTP communication using TLS (Transport Layer Security). HTTPS commonly uses TCP port 443.

During the practical, the command curl -I https://example.com was used to generate an HTTPS request. Wireshark was then used to inspect the captured traffic with the tls display filter. The tcp.port == 443 filter was also used to examine TCP packets using the standard HTTPS port.

Observations

- HTTPS traffic was generated by sending a request to example.com.
- Wireshark was used to examine available TLS packet information.
- Depending on the packets captured, TLS handshake messages and encrypted application data may be visible.
- HTTPS protects application data through encryption, so the webpage's contents generally cannot be read directly from an ordinary packet capture.

Evidence: Figure 5 shows the HTTPS/TLS traffic inspected during the practical.



Note: The exact TLS messages and packet details should be reported according to the actual screenshot and saved capture file.

8. Findings and Learning Outcomes

The practical helped develop an understanding of network traffic capture and protocol analysis using Wireshark.

Key Findings

1. Network interfaces and IP addresses can be inspected using Kali Linux terminal commands.
2. Connectivity can be tested using the ping command.
3. Wireshark can capture packets transmitted through a selected network interface.
4. Display filters can help identify particular protocols and traffic patterns.
5. HTTP and HTTPS use different security mechanisms, with HTTPS protecting application data through TLS encryption.
6. Packet captures can be saved in PCAPNG format for later analysis.

Skills Gained

- Basic network troubleshooting
- Network packet capture
- Wireshark display filtering
- HTTP and TLS traffic inspection
- Technical evidence collection and documentation

9. Conclusion

This practical introduced fundamental networking concepts and demonstrated how Wireshark can be used to capture and analyze network traffic. Kali Linux commands were used to inspect network configuration and test connectivity, while Wireshark was used to examine HTTP and HTTPS/TLS traffic.

The activity improved practical understanding of network protocols, packet filtering, and the role of encryption in protecting data during communication. The screenshots and saved packet capture provide supporting evidence of the work performed during Week 2 of the cybersecurity internship.

10. References

1. Wireshark Foundation. *Wireshark User's Guide*. https://www.wireshark.org/docs/
2. Cisco Networking Academy. *Networking Fundamentals and Introduction to Cybersecurity*. https://www.netacad.com/
3. MDN Web Docs. *An overview of HTTP*. https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
4. curl. *curl command-line tool documentation*. https://curl.se/docs/
5. Internet Engineering Task Force (IETF). *The Transport Layer Security (TLS) Protocol*. https://www.rfc-editor.org/